



Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)



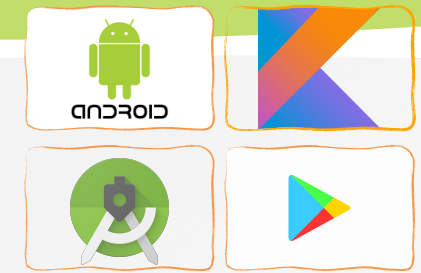
Content

» Resources

» Fonts

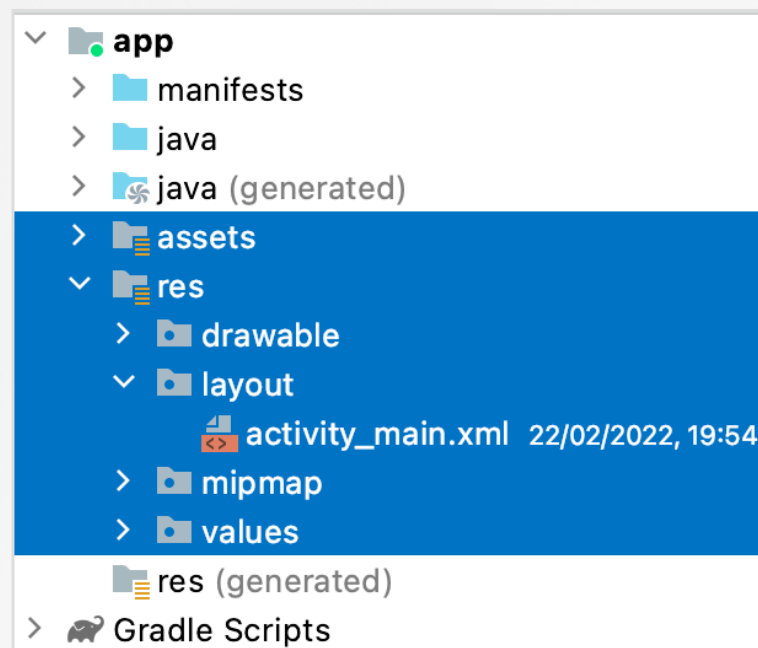
» Files

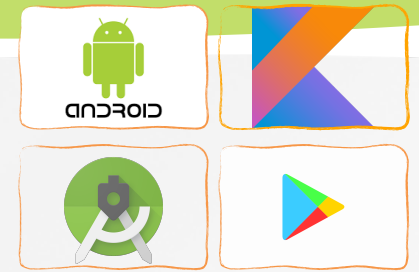
» Preferences



Resources

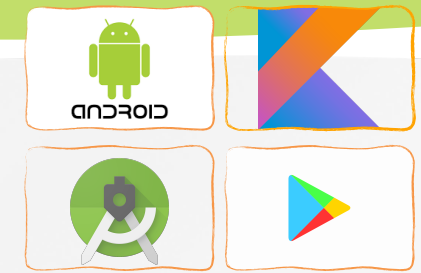
- » Resources are external files which are integrated into the application at compile time for using at execution time
 - images, strings, files, audio, animations, layouts, styles, xml ...





Resources

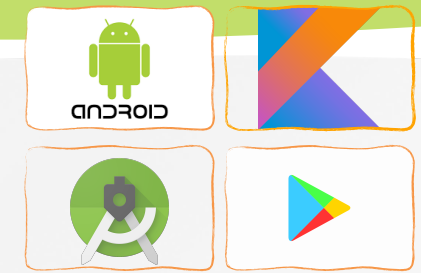
- » Resources are compiled into the final APK file.
- » Android creates **R**, a wrapper class, that we can use to refer to the resources in **/res**
- » We place our resources under the appropriate **/res** or **/assets** subdirectory in our project.
 - Copy/Paste resources we want to use into that subdirectory



Resources

<https://developer.android.com/guide/topics/resources/providing-resources.html>

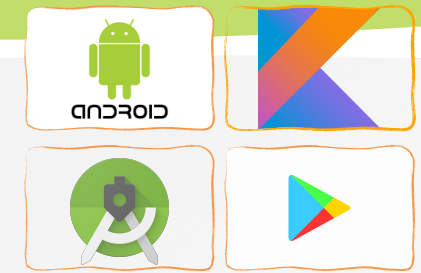
Directory	Resource Type
animator/	XML files that define property animations .
anim/	XML files that define tween animations . (Property animations can also be saved in this directory, but the <code>animator/</code> directory is preferred for property animations to distinguish between the 2 types.)
color/	XML files that define a state list of colors.
drawable/	Bitmap files (.png, .9.png, .jpg, .gif) or XML files that are compiled into the drawable resource (Bitmap files; Shapes...)



Resources

<https://developer.android.com/guide/topics/resources/providing-resources.html>

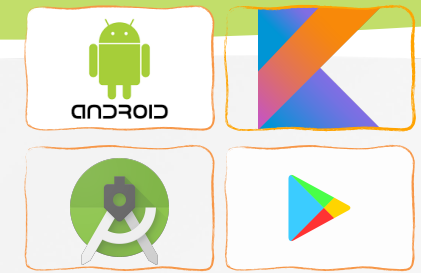
Directory	Resource Type
values/	XML files that contain simple values, such as strings, integers, and colors. Files in the values/ directory describe multiple resources. For a file in this directory, each child of the <resources> element defines a single resource. For example, a <string> element creates an R.string resource and a <color> element creates an R.color resource. Some filename conventions for resources you can create in this directory: • colors.xml for color values • strings.xml for string values. • styles.xml for styles. • arrays.xml for resource arrays (typed arrays); dimens.xml for dimension values.



Resources

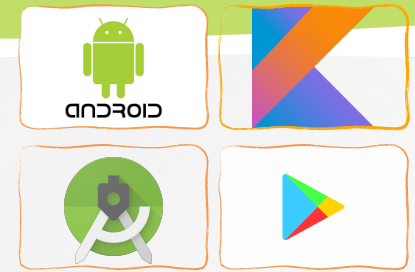
<https://developer.android.com/guide/topics/resources/providing-resources.html>

Directory	Resource Type
mipmap/	Drawable files for different launcher icon densities.
layout/	XML files that define a user interface layout.
menu/	XML files that define app menus, such as an Options Menu, Context Menu, or Sub Menu
xml/	Arbitrary XML files that can be read at runtime by calling Resources.getXML() .



Resources

Directory	Resource Type
font/	Font files (.ttf, .otf, or .ttc) or XML files that include a <code><font-family></code> element.
raw/	Arbitrary files to save in their raw form. Open these resources with a raw InputStream, call Resources.openRawResource() with the resource ID (<code>R.raw.filename</code>). If you need access to original file names and file hierarchy, you might consider saving some resources in the <i>assets/</i> directory (instead of <code>res/raw/</code>). Files in <code>assets/</code> aren't given a resource ID, so you can read them only using AssetManager.



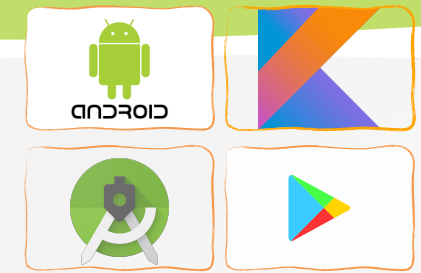
Resources

Tween animation

- » ***Tween animation*** calculates the animation with information such as the start point, end point, size, rotation, and other common aspects of an animation
- » ***hyperspace_jump.xml***: stretch, then simultaneously spin and rotate a View object

```
<set android:shareInterpolator="false">
  <scale
    android:interpolator="@android:anim/accelerate_decelerate_interpolator"
    android:fromXScale="1.0"
    android:toXScale="1.4"
    android:fromYScale="1.0"
    android:toYScale="0.6"
    android:pivotX="50%" android:pivotY="50%"
    android:fillAfter="false"
    android:duration="700" />
  <set android:interpolator="@android:anim/decelerate_interpolator">
    <scale
      android:fromXScale="1.4"
      android:toXScale="0.0"
      android:fromYScale="0.6"
      android:toYScale="0.0"
      android:pivotX="50%" android:pivotY="50%"
```

```
    android:startOffset="700"
    android:duration="400"
    android:fillBefore="false" />
    <rotate
      android:fromDegrees="0"
      android:toDegrees="-45"
      android:toYScale="0.0"
      android:pivotX="50%"
      android:pivotY="50%"
      android:startOffset="700"
      android:duration="400" />
    </set>
  </set>
```



Resources

Animation

» The animation defined in the XML will combine two different animations that happen **simultaneously** on the target view

» ***property_animator.xml***

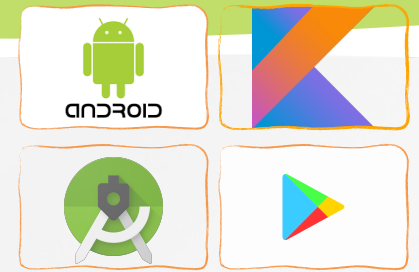
```
//together: default
<set xmlns:android="http://schemas.android.com/apk/res/android">
  <objectAnimator
    android:duration="1000"
    android:propertyName="rotation"
    android:valueFrom="0"
    android:valueTo="360" />
  <objectAnimator
    android:propertyName="alpha"
    android:duration="1000"
    android:valueTo="0.3f" />
</set>
```

<set... android:ordering="sequentially">

<set... android:ordering="together">

```
//Kotlin
(AnimatorInflater.loadAnimator(myContext, R.animator.property_animator) as AnimatorSet).apply {
    setTarget(yourView)
    start()
}
```

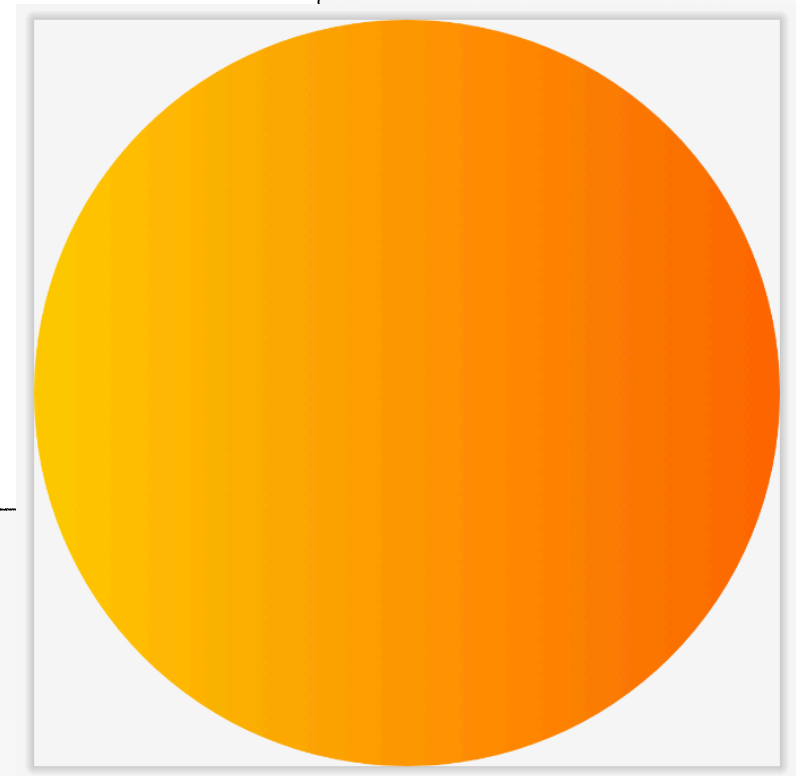
Resources shape tag

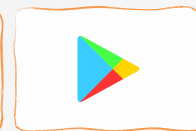


```
<shape xmlns:android="http://schemas.android.com/apk/res/
android"
    android:dither="true"
    android:shape="oval" >

    <gradient
        android:endColor="#ffff6600"
        android:gradientRadius="150"
        android:startColor="#ffffcc00"
        android:type="linear"/>

    <size
        android:height="50dp"
        android:width="50dp" />
</shape>
```

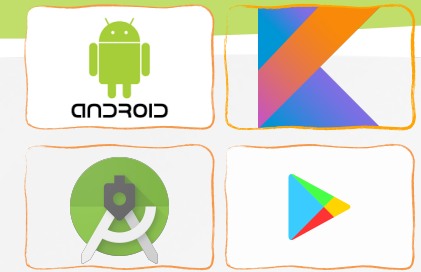




Resources

Get resources with code

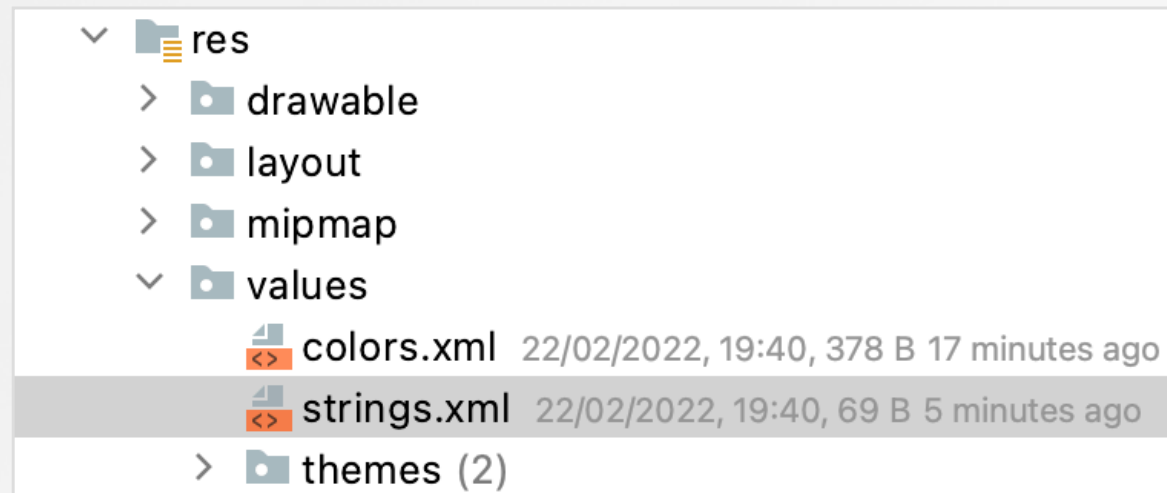
```
resources.assets.open("fileName")  
resources.assets.openXmlResourceParser("fileName")  
  
resources.getAnimation(id)  
resources.getColor(id)  
resources.getDrawable(id)  
resources.getIntArray(id)  
resources.getStringArray(id)  
resources.getTextArray(id)  
resources.getLayout(id)  
resources.getMovie(id)  
resources.getString(id)  
resources.xml(id)
```

Resources

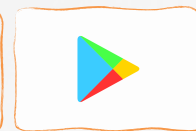
Get resources with code

» Getting **String** resources from: **res/values/strings.xml**



```
<resources>
    <string name="app_name">FontApp</string>
</resources>
```

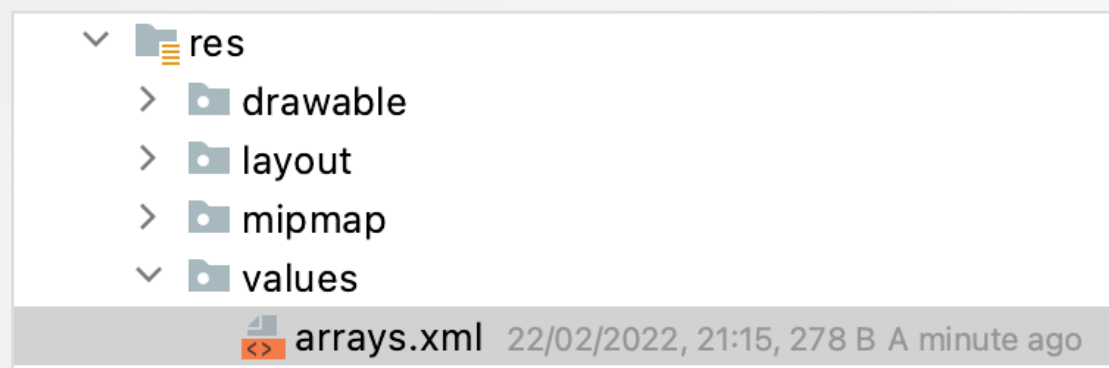
```
textView.setText(getString(R.string.app_name))
```



Resources

Get resources with code

» Getting array resources from: **res/values/arrays.xml**



```
val colors = resources.getStringArray(R.array.colors)
```

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string-array name="colors">
    <item>red</item>
    <item>orange</item>
    <item>yellow</item>
    <item>green</item>
    <item>blue</item>
  </string-array>
</resources>
```



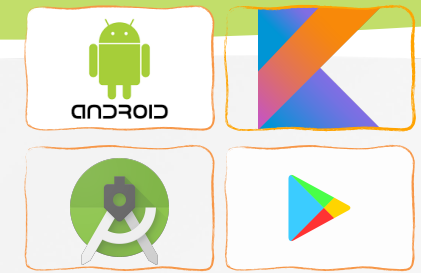
Content

» Resources

» **Fonts**

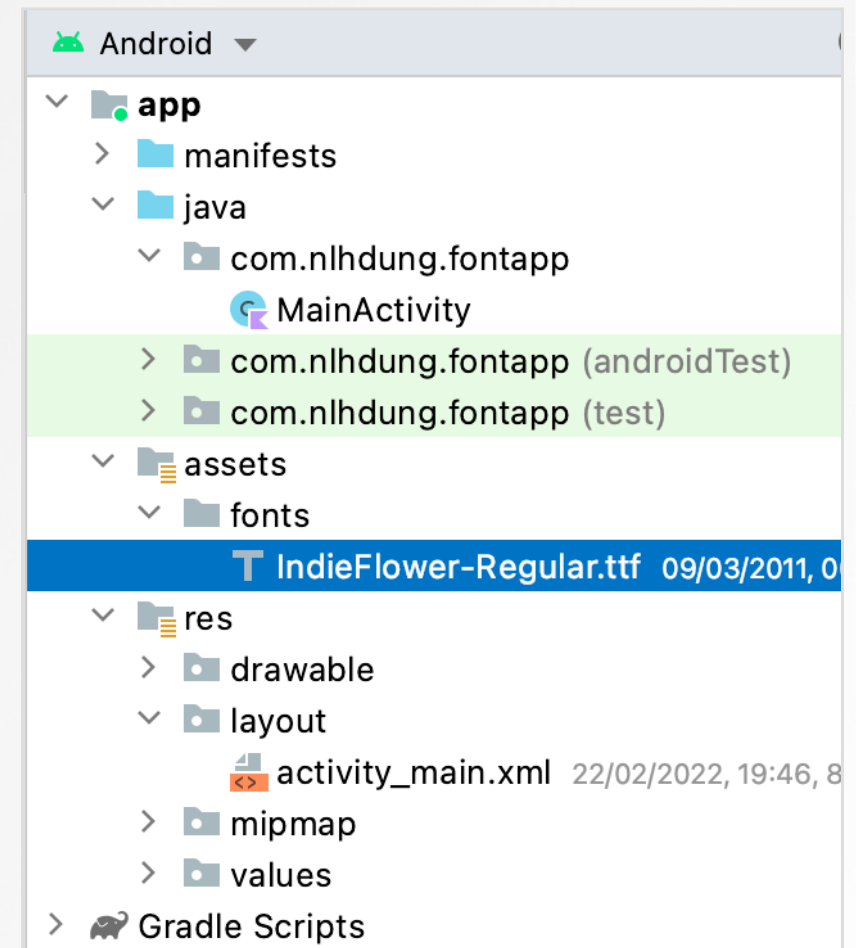
» Files

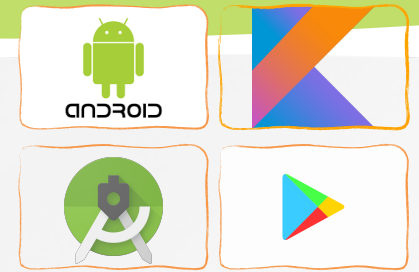
» Preferences



Fonts

- » We can add any font to our application by following these steps:
- Create the **/fonts** folder in the **/assets** directory.
 - Copy any fonts we want to use into the new folder
 - Use Kotlin/Java code to bind the font with the UI widget wanting to display the custom typeface





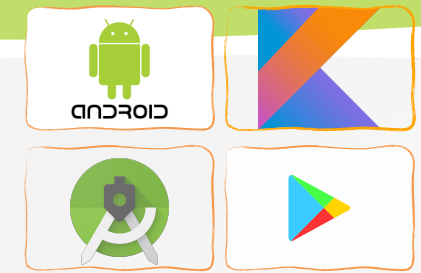
Fonts

» Main activity:

```
val textView : TextView = findViewById(R.id.textView)
val typeface = Typeface.createFromAsset(assets,
    "fonts/IndieFlower-Regular.ttf")
textView.setTextSize(40F)
textView.setTypeface(typeface)
```

Default typeface

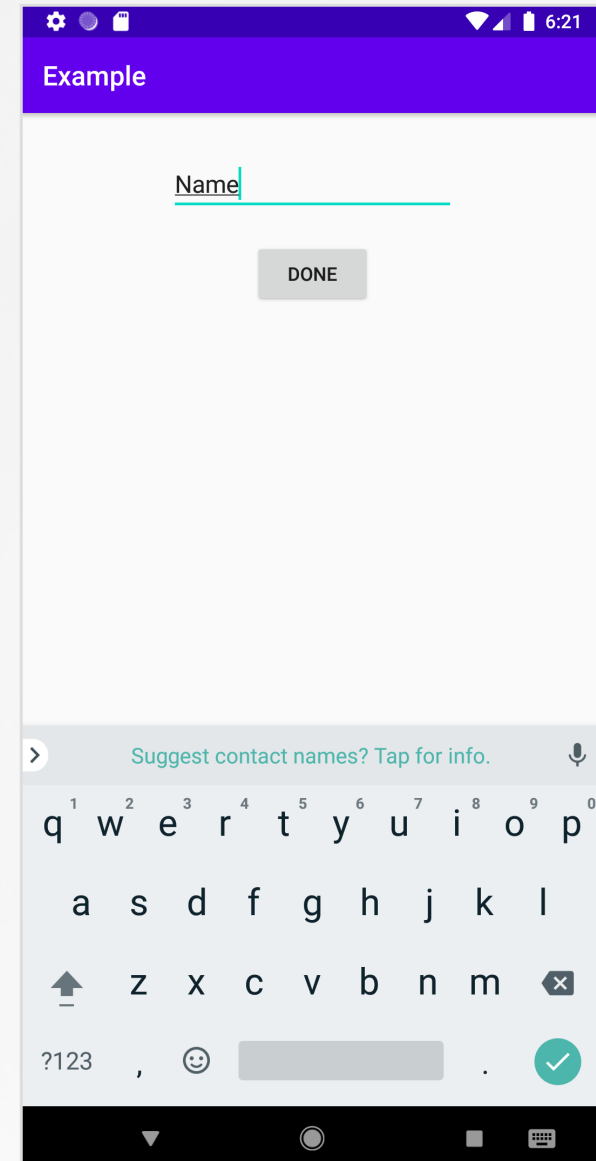
New typeface



Graphical User Interfaces (cont.)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/constraintLayout"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/editText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"
        android:ems="10"
        android:inputType="textPersonName"
        android:text="Name"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Done"
        app:layout_constraintEnd_toEndOf="@+id/editText"
        app:layout_constraintStart_toStartOf="@+id/editText"
        app:layout_constraintTop_toBottomOf="@+id/editText" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

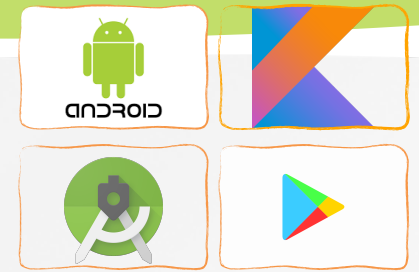


Actual UI displayed by the app 18



Content

- » Resources
- » **Fonts**
- » **Files**
- » Preferences



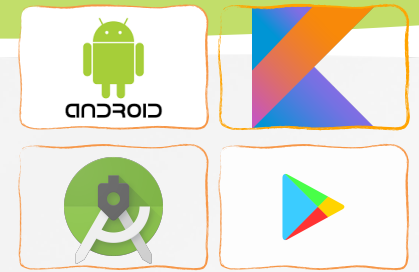
Data and file storage overview

» Android uses a file system that's similar to disk-based file systems on other platforms.

<https://developer.android.com/training/data-storage>

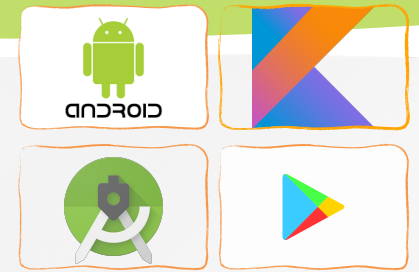
– App-specific storage:

- Store files that are meant for your app's use only, either in dedicated directories within an **internal storage** volume or different dedicated directories within **external storage**.
- Use the directories within internal storage to save sensitive information that other apps shouldn't access.



Data and file storage overview

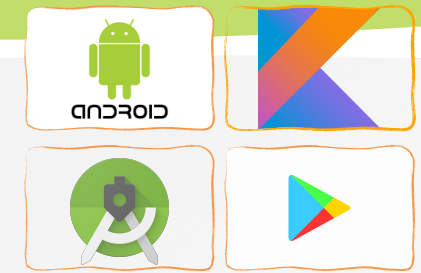
- **Shared storage:** Store files that your app intends to share with other apps, including media, documents, and other files.
- **Preferences:** Store private, primitive data in key-value pairs.
- **Databases:** Store structured data in a private database using the Room persistence library.



Files

- » Files can be stored **internally** in the device's main memory (small) or **externally** in the SD card (larger).
- » Apps themselves are stored within internal storage by default.
 - If our APK size is very large, we can indicate a preference within our app's manifest file to install our app on external storage:

```
<manifest ...  
  android:installLocation="preferExternal">  
  ...  
</manifest>
```

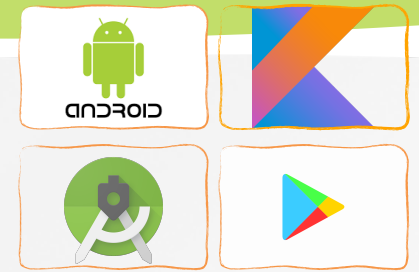


Device File Explorer

Internal
main memory

External
SD card

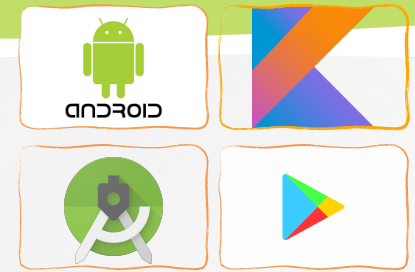
Device File Explorer		
Emulator Pixel_5_API_28 Android 9, API 28		
Name	Permissions	Date
> acct	dr-xr-xr-x	2022-02-21 22:24
> bin	lrw-r--r--	2020-08-05 08:16
> bugreports	lrw-r--r--	2020-08-05 08:16
> cache	drwxrwx---	2020-08-05 07:29
> config	drwxr-xr-x	2022-02-21 22:24
> d	lrw-r--r--	2020-08-05 08:16
> data	drwxrwx--x	2022-01-21 11:44
> dev	drwxr-xr-x	2022-02-21 22:24
> etc	lrw-r--r--	2020-08-05 08:16
> lost+found	drwx-----	2020-08-05 08:16
> metadata	drwxr-xr-x	2020-08-05 07:29
▼ mnt	drwxr-xr-x	2022-02-21 22:24
> appfuse	drwx--x--x	2022-02-21 22:24
> asec	drwxr-xr-x	2022-02-21 22:24
> expand	drwxrwx--x	2022-02-21 22:24
> media_rw	drwxr-x---	2022-02-21 22:24
> obb	drwxr-xr-x	2022-02-21 22:24
> runtime	drwx-----	2022-02-21 22:24
> sdcard	lrwxrwxrwx	2022-02-21 22:24
> secure	drwx-----	2022-02-21 22:24
> user	drwxr-xr-x	2022-02-21 22:24
> vendor	drwxr-xr-x	2022-02-21 22:24
> odm	drwxr-xr-x	2020-08-05 07:29



Resource Files

- » Using Android Resource Files such as
 - res/drawable , res/raw, res/menu, res/style...
- » Resource files could be accessed through the ***.getResources()***

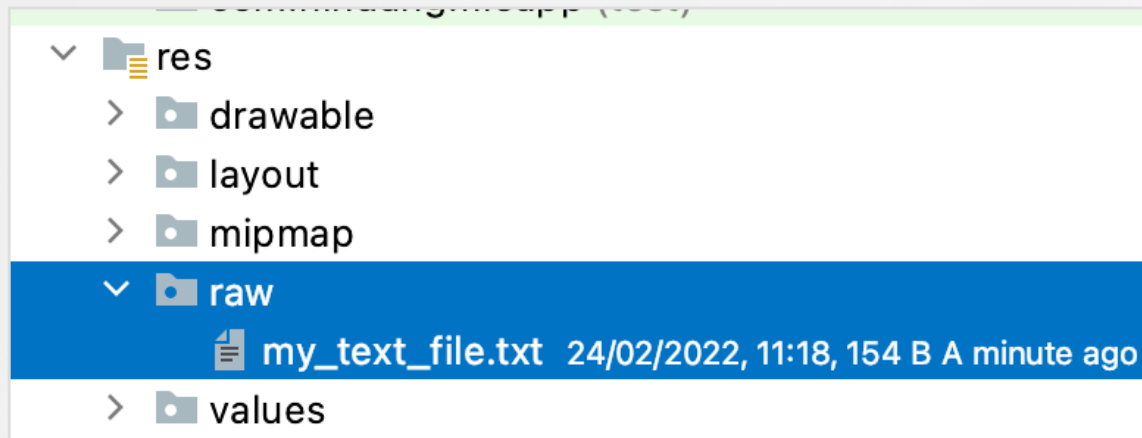
```
val iStream: InputStream =  
    resources.openRawResource(R.drawable.myFile)
```

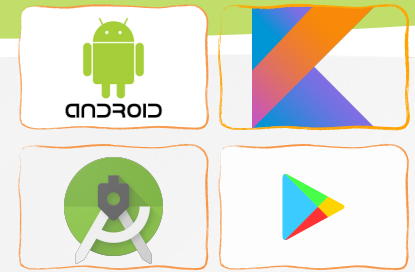


Resource Files

Example 1: read resource file

- » New > Folder > Raw resources folder
 - Add txt file ***my_text_file.txt*** into that folder





Resource Files

Example 1: read resource file

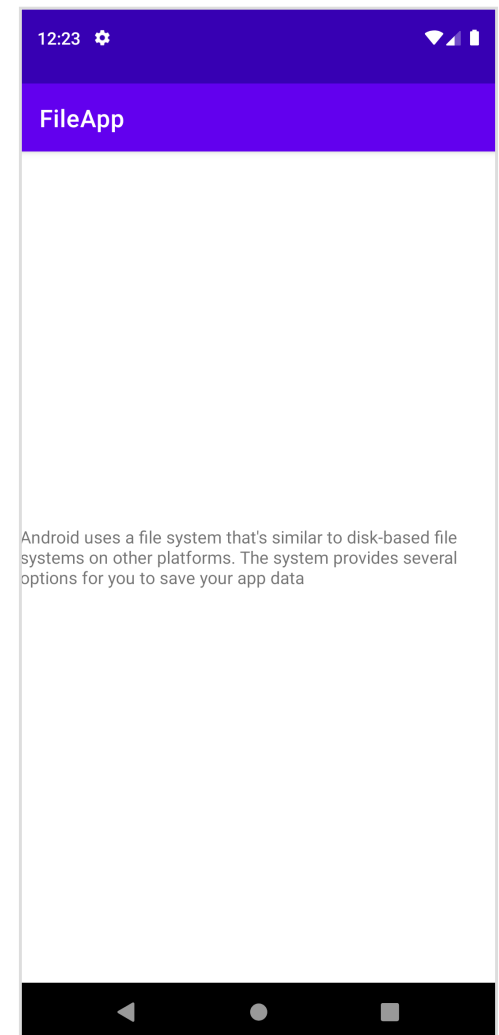
» Read file's content in Main Activity

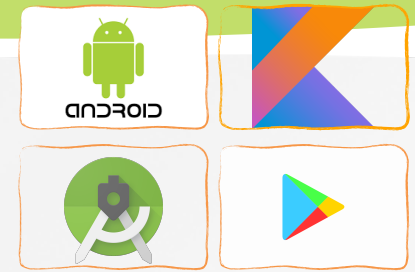
```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        var txtMsg = findViewById<View>(R.id.textView) as TextView
        try {
            txtMsg.setText(PlayWithRawFiles())
        } catch (e: IOException) {
            txtMsg.text = "Problems: " + e.message
        }
    }

    @Throws(IOException::class)
    fun PlayWithRawFiles() : String{
        val buf = StringBuffer()
        val iStream: InputStream = resources.openRawResource(R.raw.my_text_file)
        val reader = BufferedReader(InputStreamReader(iStream))

        if (iStream != null) {
            var line : String? = reader.readLine()
            while (line != null) {
                buf.append(line + "\n")
                line = reader.readLine()
            }
        }
        iStream.close()
        return buf.toString()
    }
}
```

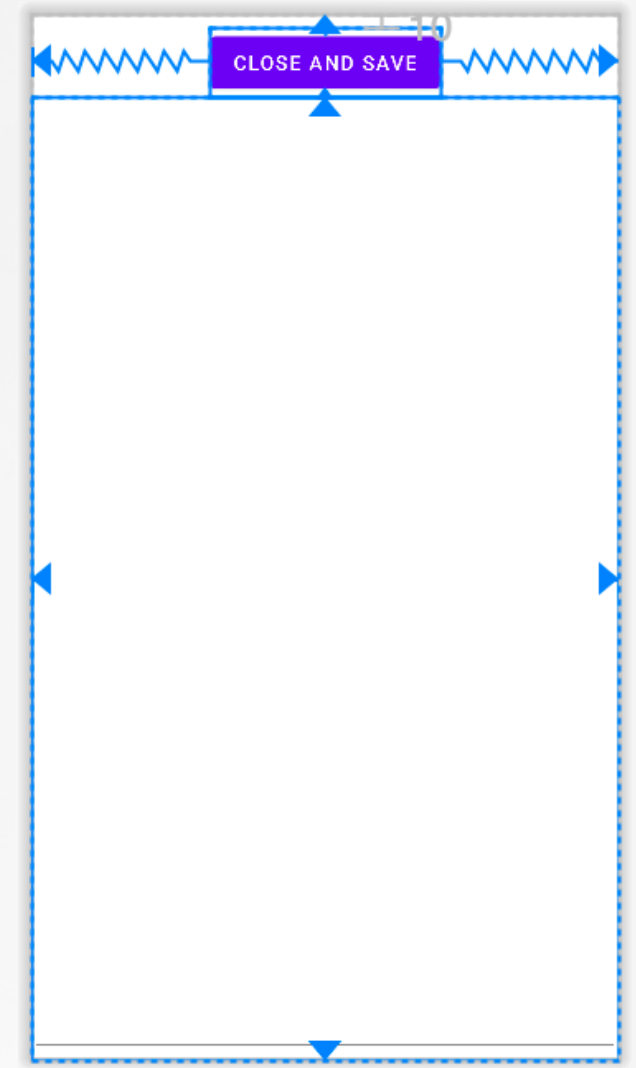


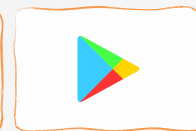


Example 2

Read/Write an Internal File

- » The application will collect data from the UI and save it to a persistent data file into the internal Android memory.
- Delete file
 - Read file's content onStart
 - Save file's content onPause



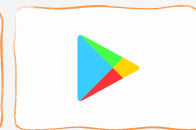


Example 2

Read/Write an Internal File

» Main Activity : editText, fileName and button event

```
class MainActivity : AppCompatActivity() {  
    val fileName = "test01.txt"  
    lateinit var editText: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        editText = findViewById(R.id.editText)  
  
        val btnFinish: Button = findViewById(R.id.saveBtn)  
        btnFinish.setOnClickListener {  
            finish()  
        }  
    }  
    ...  
}
```

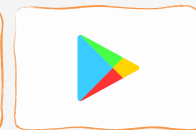



Example 2

Read/Write an Internal File

» Main Activity: reading onStart()

```
override fun onStart() {  
    super.onStart()  
    try {  
        val inputStream: InputStream? = openFileInput(fileName)  
        if (inputStream != null) {  
            val inputStreamReader = InputStreamReader(inputStream)  
            val reader = BufferedReader(inputStreamReader)  
            val stringBuffer = StringBuffer()  
  
            var line : String? = reader.readLine()  
            while (line != null) {  
                stringBuffer.append(line + "\n")  
                line = reader.readLine()  
            }  
  
            inputStream.close()  
            editText.setText(stringBuffer.toString())  
        }  
    } catch (e: FileNotFoundException) {  
    } catch (t: Throwable) {  
        Toast.makeText(this, "Exception: $t", Toast.LENGTH_SHORT).show()  
    }  
}
```

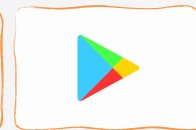


Example 2

Read/Write an Internal File

» Main Activity: saving onPause()

```
// onPause
override fun onPause() {
    super.onPause()
    try {
        //File will be in "/data/data/$packageName/files/"
        val out = OutputStreamWriter(openFileOutput(fileName, 0))
        out.write(editText.getText().toString())
        out.close()
    } catch (t: Throwable) {
        editText.setText(t.message)
    }
}
```

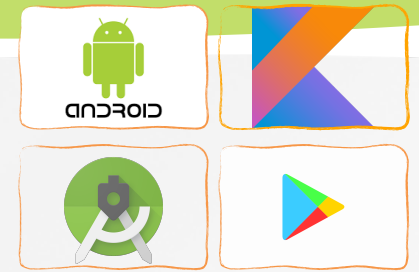


Example 2

Read/Write an Internal File

» Main Activity: deleting file

```
private fun deleteFile() {  
    val packageName = applicationContext.packageName  
    val path = "/data/data/$packageName/files/$fileName"  
    val f1 = File(path)  
    Toast.makeText(applicationContext, "Exists " + f1.exists(),  
        Toast.LENGTH_SHORT).show()  
    val success: Boolean = f1.delete()  
    if (!success) {  
        Toast.makeText(applicationContext, "Deletion failed.",  
            Toast.LENGTH_SHORT).show()  
    } else {  
        Toast.makeText(applicationContext, "OK. File deleted.",  
            Toast.LENGTH_SHORT).show()  
    }  
}
```

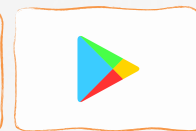


Example 3

Reading/Writing to the Device's **SD card**

» We need to request permission to read and write to the SD card by adding the following clauses to our `AndroidManifest.xml`

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

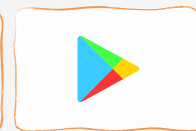


Example 3

Reading/Writing to the Device's **SD card**

» Main activity: fileName, editText, button's event, requesting permission...

```
class MainActivity : AppCompatActivity() {  
    val fileName = "test01.txt"  
    lateinit var editText: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        editText = findViewById(R.id.editText)  
  
        val btnFinish: Button = findViewById(R.id.saveBtn)  
        btnFinish.setOnClickListener {  
            if (ContextCompat.checkSelfPermission(  
                applicationContext, Manifest.permission.WRITE_EXTERNAL_STORAGE)  
                != PackageManager.PERMISSION_GRANTED  
            ) {  
                val permissions = arrayOf(Manifest.permission.READ_EXTERNAL_STORAGE,  
                    Manifest.permission.WRITE_EXTERNAL_STORAGE)  
                ActivityCompat.requestPermissions(this, permissions, 11111);  
            } else {  
                finish()  
            }  
        }  
    }  
}
```

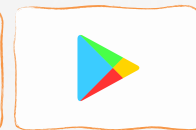


Example 3

Reading/Writing to the Device's **SD card**

» Check permission result

```
override fun onRequestPermissionsResult(requestCode: Int,
    permissions: Array<out String>, grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    when (requestCode) {
        11111 -> {
            // If request is cancelled, the result arrays are empty.
            if (grantResults.size > 0
                && grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {
                finish() // finish and save file's content
            } else {
                Toast.makeText(this, "permission denied", Toast.LENGTH_SHORT).show()
            }
            return
        }
    }
}
```



Example 3

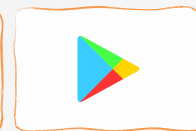
Reading/Writing to the Device's **SD card**

» Read file's content

```
fun readFile(){
    try {
        val path = "mnt/sdcard/$fileName"
        val file = File(path)
        val inputStream: InputStream? = FileInputStream(file)
        if (inputStream != null) {
            val inputStreamReader = InputStreamReader(inputStream)
            val reader = BufferedReader(inputStreamReader)
            val stringBuffer = StringBuffer()

            var line : String? = reader.readLine()
            while (line != null) {
                stringBuffer.append(line + "\n")
                line = reader.readLine()
            }

            inputStream.close()
            reader.close()
            editText.setText(stringBuffer.toString())
        }
    } catch (e: FileNotFoundException) {
    } catch (t: Throwable) {
        Toast.makeText(this, "Exception: $t", Toast.LENGTH_SHORT).show()
    }
}
```



Example 3

Reading/Writing to the Device's **SD card**

» Create file and write file's content

```
fun createandWriteFile(){
    try {
        val path = "mnt/sdcard/$fileName"
        val file = File(path)
        file.createNewFile()
        val outputStream = FileOutputStream(file)
        val writer = OutputStreamWriter(outputStream)

        writer.write(editText.getText().toString())

        writer.close()
        outputStream.close()
    } catch (t: Throwable) {
        editText.setText(t.message)
    }
}
```



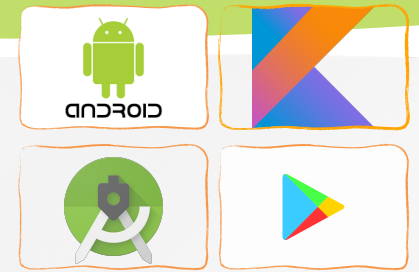

Content

» Resources

» **Fonts**

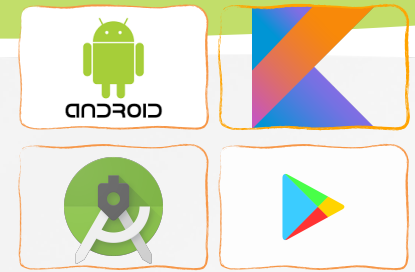
» Files

» **Preferences**



Preferences

- » It is an Android mechanism to store and retrieve **Key-Value** pairs of primitive data types
- » We can use Preferences to keep state information and shared data among several activities of an application.
- » In each entry of the form **key-value**
 - The key is a string
 - The value must be a primitive data type.



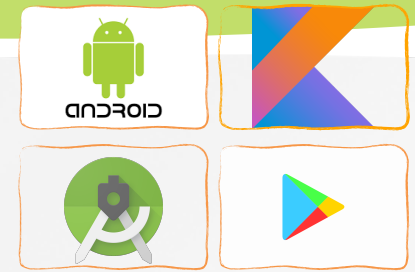
Preferences

» The path of your preferences

`/data/data/$packageName/Shared_prefs/prefFileName.xml`

» Get a handle to shared preferences

- ***getSharedPreferences()***: use this if you need multiple shared preference files identified by name, which you specify with the first parameter. You can call this from any Context in your app.
- ***getPreferences()***: use this from **an Activity** if you need to use only one shared preference file for the activity.
 - This retrieves a default shared preference file that belongs to the activity, you don't need to supply a name.



Example 4

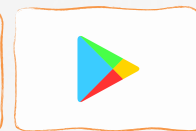
Preferences

» Save and read preferences

```
val mode = Activity.MODE_PRIVATE
val mySharedPreferences = getSharedPreferences("MYPREFS", mode)

val myEditor = mySharedPreferences.edit()
myEditor.putString("stdName", "Dung")
myEditor.putInt("stdAge", 21)
myEditor.putFloat("stdPoint", 1500000.00f)
myEditor.putString("stdBirthDate", Date().toLocaleString())
myEditor.commit()

//extract the <key/value> pairs, use default param for missing data
val stdName = mySharedPreferences.getString("stdName", "defNameValue")
val stdAge = mySharedPreferences.getInt("stdAge", 18)
```



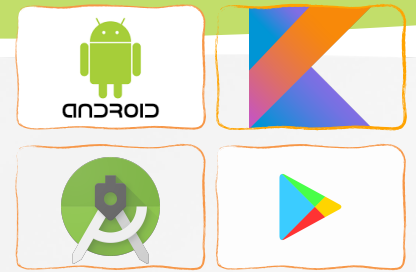
Example 3

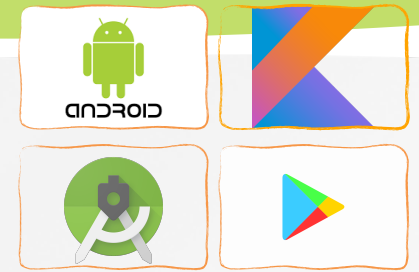
Reading/Writing to the Device's **SD card**

» /data/data/com.nlhdung.fileapp/
shared_prefs/***MYPREFS.xml***

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="stdBirthDate">Feb 24, 2022 11:09:20 PM</string>
  <int name="stdAge" value="21" />
  <float name="stdPoint" value="1500000.0" />
  <string name="stdName">Dung</string>
</map>
```

Q&A



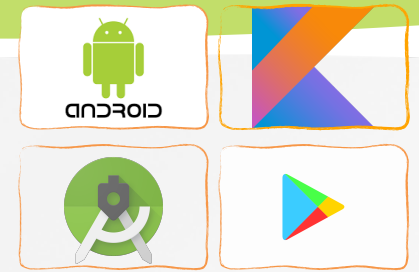


Practice 1

- » Let's do all above examples for implementation:
- Resources (image, strings, arrays...)
 - Add and change fonts for an activity
 - Read and write internal files (+EditText...)
 - Read and write external files (+EditText...)
 - Preferences

Appendix A

Using pictures in the SD card

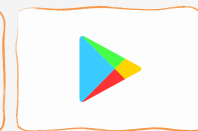


» Request permission

```
val EXTERNAL_PERMS = arrayOf(
    Manifest.permission.WRITE_EXTERNAL_STORAGE,
    Manifest.permission.READ_EXTERNAL_STORAGE
)
val EXTERNAL_REQUEST_CODE = 113

//android version 23 or higher, you need to give run time permission programmatically i
fun requestPermission(): Boolean {
    var isPermissionOn = true
    if (Build.VERSION.SDK_INT >= 23) {
        if (!canAccessExternalSd()) {
            isPermissionOn = false
            requestPermissions(EXTERNAL_PERMS, EXTERNAL_REQUEST_CODE)
        }
    }
    return isPermissionOn
}

fun canAccessExternalSd(): Boolean {
    return PackageManager.PERMISSION_GRANTED ==
        ContextCompat.checkSelfPermission(this,
            Manifest.permission.WRITE_EXTERNAL_STORAGE)
}
```

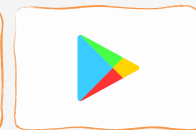



Appendix A

Using pictures in the SD card

» Request permission

```
override fun onRequestPermissionsResult( requestCode: Int,  
permissions: Array<out String>, grantResults: IntArray) {  
    when (requestCode) {  
        EXTERNAL_REQUEST_CODE -> {  
            // If request is cancelled, the result arrays are empty.  
            if ((grantResults.isNotEmpty() && grantResults[0] ==  
                PackageManager.PERMISSION_GRANTED)) {  
                useSDCard()  
            } else {  
                requestPermissions(EXTERNAL_PERMS, EXTERNAL_REQUEST_CODE)  
            }  
            return  
        }  
        // Add other 'when' lines to check for other  
        // permissions this app might request.  
        else -> {  
            // Ignore all other requests.  
        }  
    }  
}
```

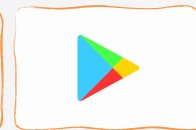


Appendix A

Using pictures in the SD card

» Check permission and read pictures

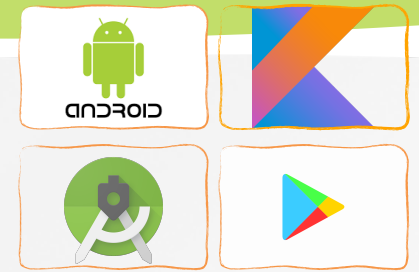
```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    if (requestForPermission()) {  
        readPicsFromSDCard()  
    }  
}
```



APPENDIX B. Writing/Reading file using Scanner and PrintWriter classes

» Create file and write file's content

```
fun createAndWriteWithPrintWriter(){
    try {
        val sdCardPath = Environment.getExternalStorageDirectory().getPath()
        val mySDFileName = "$sdCardPath/$fileName"
        val outfile = PrintWriter(FileWriter(mySDFileName))
        outfile.println("Test: line 1")
        outfile.println("Test: line 2")
        outfile.close()
    } catch (e: FileNotFoundException) {
        Toast.makeText(this, "Exception: $e", Toast.LENGTH_SHORT).show()
    } catch (t: Throwable) {
        Toast.makeText(this, "Exception: $t", Toast.LENGTH_SHORT).show()
    }
}
```



Reference

- » <https://kotlinlang.org/docs/reference/>
- » Peter Späth, Learn Kotlin for Android Development, 2019
- » Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- » Peter Späth, Pro Android with Kotlin, 2019
- » Milos Vasic, Mastering Android Development with Kotlin, 2017
- » <https://developer.android.com/training/permissions/requesting>
- » <https://developer.android.com/guide/>